University of Applied Sciences and Arts Northwestern Switzerland
School of Computer Science

Cloud Computing
(cloud)

Module 7: Large Scale Container Orchestration (Kubernetes)

Prof. Dr. Sebastian Graf (sebastian.graf@fhnw.ch)
Norwin Schnyder (norwin.schnyder@fhnw.ch)

Input of this Module

- https://learning.oreilly.com/playlists/43827098-7a87-435e-823e-736586b5694c (07-largescalecontainerarch)
  - Brendan Burns, Joe Beda, Kelsey Hightower, Lachlan Evenson - Kubernetes: Up and Running, 3rd Edition

- Kelsey Hightower - Kubernetes The Hard Way
  - https://github.com/kelseyhightower/kubernetes-the-hard-way

- Honeypot - Kubernetes: The Documentary
  - https://www.youtube.com/watch?v=BE77h7dmoQU

- Linux Foundation Education - Introduction to Kubernetes (LFS158)
  - https://trainingportal.linuxfoundation.org/courses/introduction-to-kubernetes

Agenda

1. Introduction to Container Orchestration

2. The History of Kubernetes

3. Kubernetes Objects

4. Kubernetes Cluster Architecture (Kelsey Hightower's Kubernetes The Hard Way)

What is Container Orchestration?

Running containers on a single host for development and testing of applications may be a suitable option. However, when migrating production environments, that is no longer a viable option because the applications and services need to meet specific requirements:

- Fault-tolerance
- On-demand scalability
- Optimal resource usage
- Auto-discovery to automatically discover and communicate with each other
- Accessibility from the outside world
- Seamless updates/rollbacks without any downtime

Container orchestrators are tools which group systems together to form clusters where containers' deployment and management is automated at scale while meeting the requirements mentioned above. The clustered systems confer the advantages of distributed systems, such as increased performance, cost efficiency, reliability, workload distribution, and reduced latency.

What is Kubernetes?

"Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications."

- Kubernetes (also referred to as **k8s**) originates from Greek, meaning helmsman or pilot, we can think of Kubernetes as the pilot on a ship of containers.
- Kubernetes is highly inspired by the **Google Borg system**, a container and workload orchestrator for its global operations, Google has been using for more than a decade (since 2003/2004).
- With its v1.0 release in July **2015**, Google donated Kubernetes to the *Cloud Native Computing Foundation (CNCF)*
- In 2016, Kubernetes goes mainstream with e.g., *Pokémon GO*.
- Kubernetes is written in the Go language and licensed under the Apache License Version 2.0.

Kubernetes Distributions

Nowadays, there is a wide variety of Kubernetes distributions available, including both on-premise software solutions and cloud hosted services. They may be open-source or come with enterprise support.

KubeCon London (2016→2025)

https://www.linkedin.com/posts/cloud-native-computing-foundation_kubecon-cloudnativecon-activity-7313952932893188096-gvBL

Cloud Native Computing Foundation (CNCF)

CLOUD NATIVE COMPUTING FOUNDATION

The Cloud Native Computing Foundation (CNCF) is one of the largest sub-projects (over 300'000 contributors, 4.14B Lines of Code) hosted by the Linux Foundation. CNCF aims to accelerate the adoption of containers, microservices, and cloud-native applications by ***building a sustainable ecosystems***.

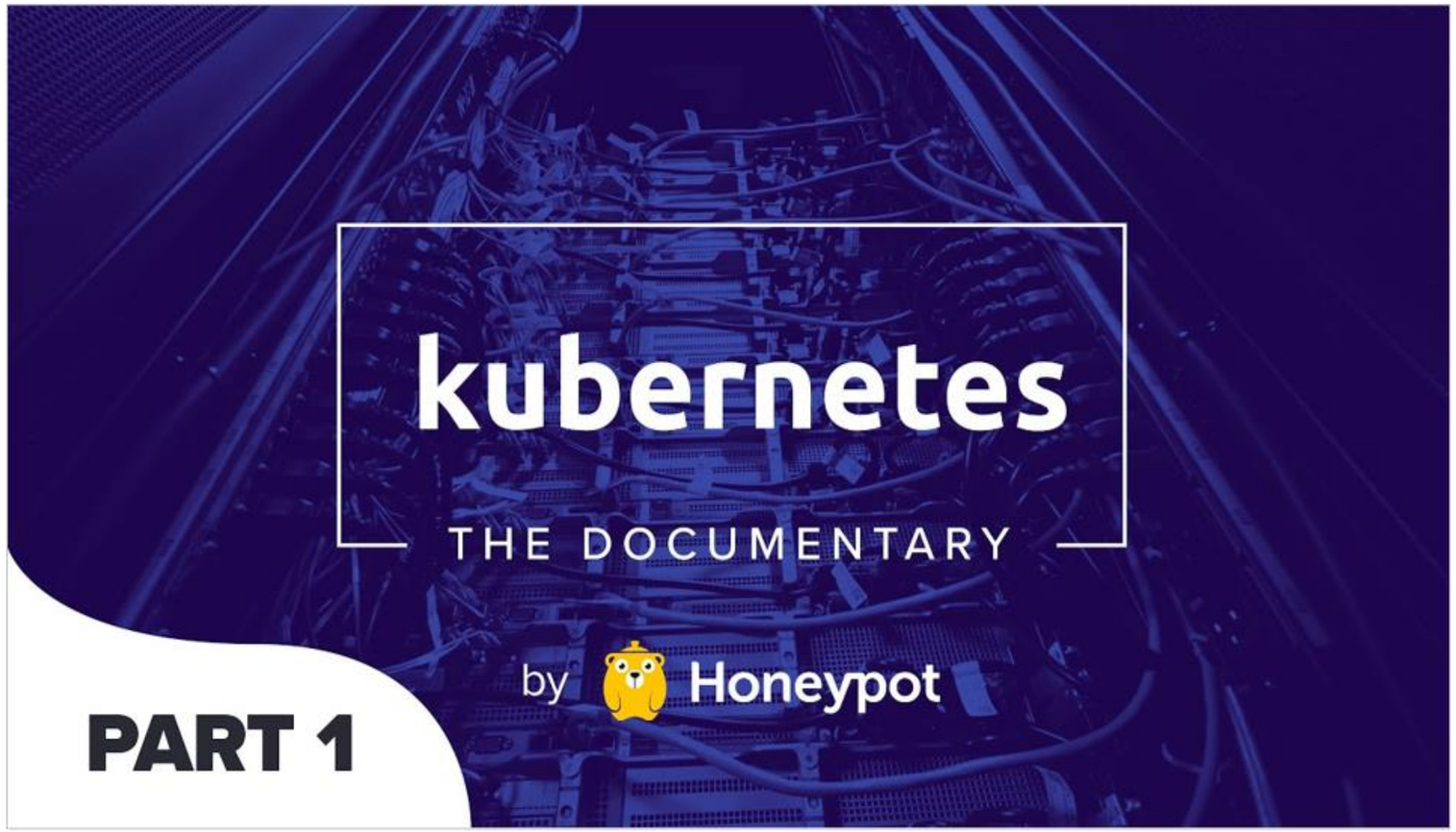
CNCF hosts a multitude of projects and provides resources (funding, technical support, governance, marketing, ...) to each of the projects, but, at the same time, each project continues to operate independently under its pre-existing governance structure and with its existing maintainers.

Explore the **CN Landscape** to learn more about all cloud native open-source projects and proprietary products:

CLOUD NATIVE LANDSCAPE

https://landscape.cncf.io/

What is Kubernetes?



Honeypot - Kubernetes: The Documentary (Part 1&2)
https://www.youtube.com/watch?v=BE77h7dmoQU

Kubernetes Objects: Record of intent

Command (imperative)

"Run Envoy Proxy with … "

```
$ docker run -d \
  -e CONFIG="/etc/envoy/envoy.yaml" \
  -p 8080:10000 -p 8081:9901 \
  envoyproxy/envoy:v1.33-latest \
  bin/sh -c "envoy -c \$CONFIG"
```

Record of intent (declarative)

"Envoy Proxy container running with … "

```
containers:
- name: envoy
  image: envoyproxy/envoy:v1.33-latest
  command:
  - bin/sh
  - -c
  - envoy -c $CONFIG
  env:
  - name: CONFIG
    value: /etc/envoy/envoy.yaml
  ports:
  - name: http
    containerPort: 10000
  - name: admin
    containerPort: 9901
```

Kubernetes Objects: Pod

The **apiVersion** specifies the version and group of the Kubernetes API and **kind** indicates the type of object.

The **metadata** contains data that helps uniquely identify the object, including a *name* string, *UID*, and optional *namespace.*

The **spec** describes the *desired state* (e.g., the container image for each container within that pod).

Note: the object **spec** format varies for each Kubernetes object. While the **status** of a Kubernetes object may differ, it typically adheres to a similar structure (conditions).

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
  namespace: default
  labels:
    environment: dev
    app: envoy
spec:
  containers:
  - name: envoy
    image: envoyproxy/envoy:v1.33-latest
status:
  conditions:
  - status: "True"
    type: Ready
  containerStatuses:
  - containerID: containerd://df7b3cf77b0dfa671
    image: docker.io/envoyproxy/envoy:v1.33-latest
```

The **status** describes the *current state* of the object, supplied and updated by the Kubernetes system and its components (controllers).

Kubernetes Objects

Kubernetes objects are **persistent entities** in the Kubernetes system. Kubernetes uses these entities to represent the state of your cluster. Those entities describe:

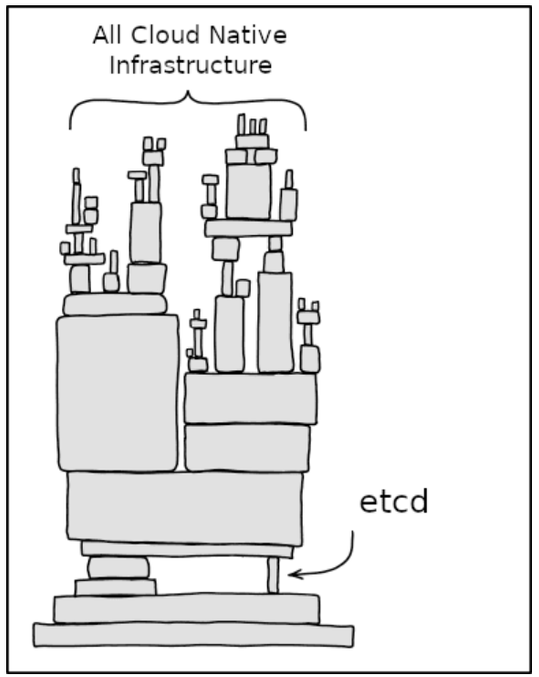
- What containerized applications are running (and on which nodes)
- The resources available to those applications
- The policies around how those applications behave (restart/upgrade policies, ingress/egress, access control, …)

A Kubernetes object is a ***record of intent*** *(describing the desired state)*, once it's created, the Kubernetes system will constantly work to ensure that the object exists.

Persistent Storage for Kubernetes Objects: etcd

etcd is a distributed reliable **key-value store** for the most critical data of a distributed system, with a focus on being:

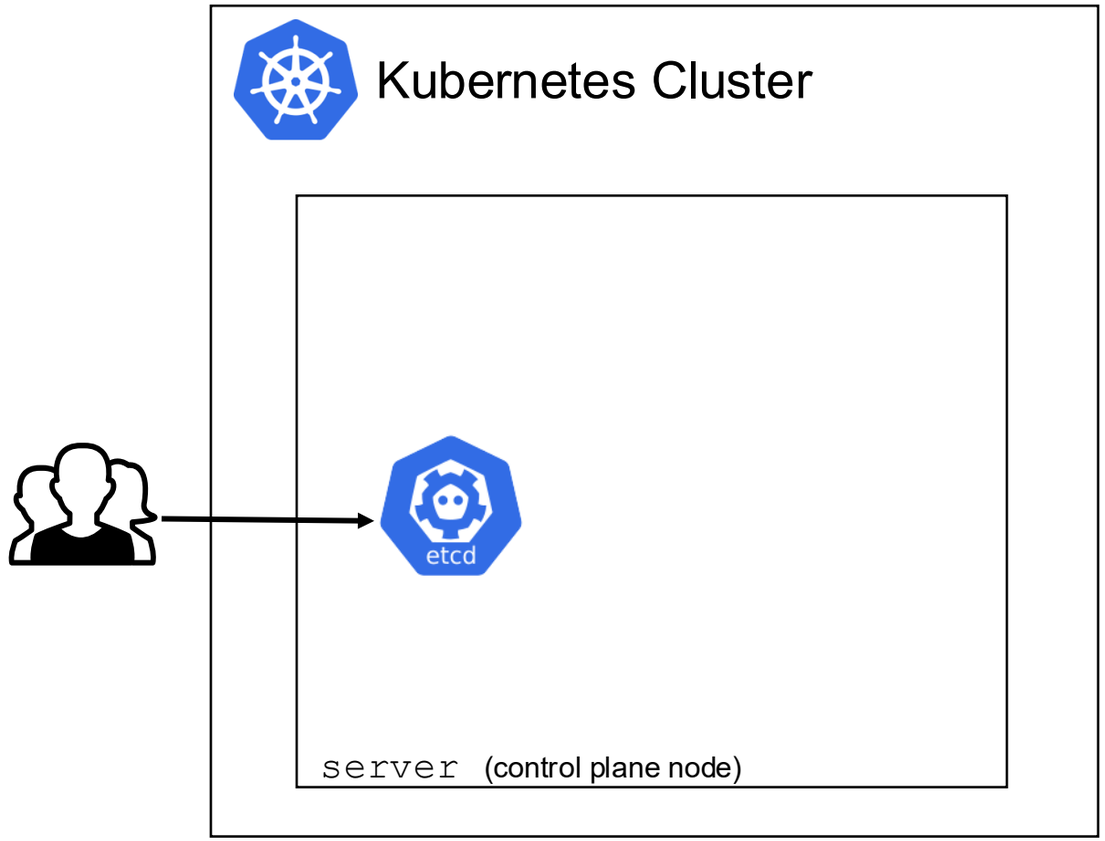
- Simple: well-defined, user-facing API (gRPC)
- Secure: automatic TLS with optional client cert authentication
- Fast: benchmarked 10,000 writes/sec
- Reliable: properly distributed using Raft



To persist the Kubernetes cluster's state, all cluster configuration data (objects) is saved in etcd. It may be deployed on the control plane node (stacked topology), or on its dedicated host (external topology) to help reduce the chances of data store loss by decoupling it from the other control plane agents.

Important: Always ensure that ectd is encrypted (as it might contain sensitive information) and a backup plan is in place (for production environments).

Kubernetes Cluster Architecture



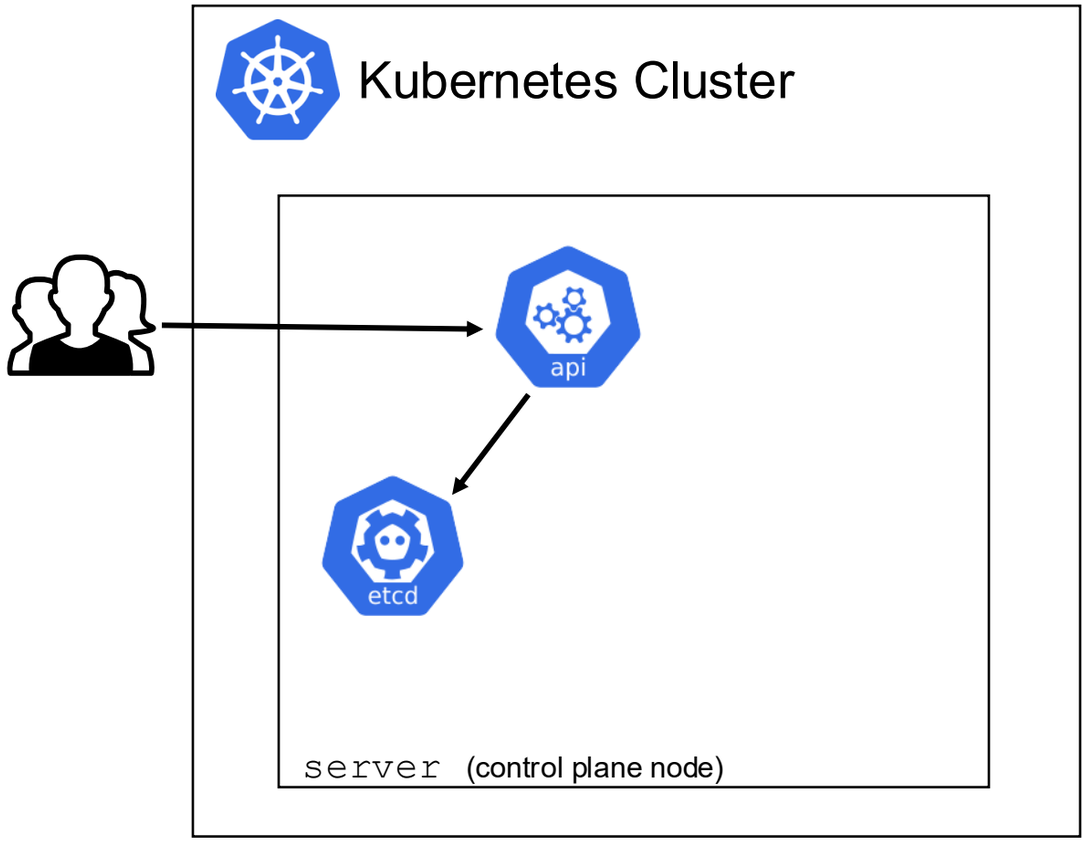
Kubernetes API Server

- The **Kubernetes API server (kube-apiserver)** is a key component of the Kubernetes control plane, serving as the frontend for the Kubernetes API.

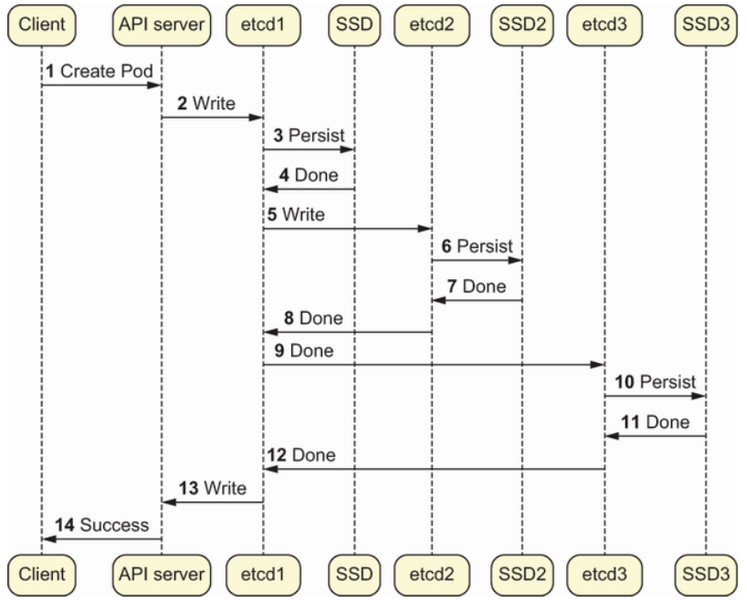
- The Kubernetes API can be accessed via the `kubectl` CLI tool, client libraries, or direct REST requests, with both human users and Kubernetes service accounts authorized for access.

- When receiving a request, the API server ensures proper authentication, authorization, and validation before persisting modifications to the etcd store.

Kubernetes Cluster Architecture



Interaction between API server and etcd
(for a horizontally scaled control plane)



Note: etcd uses the *Raft consensus algorithm* to ensure strong consistency (*CAP theorem*).

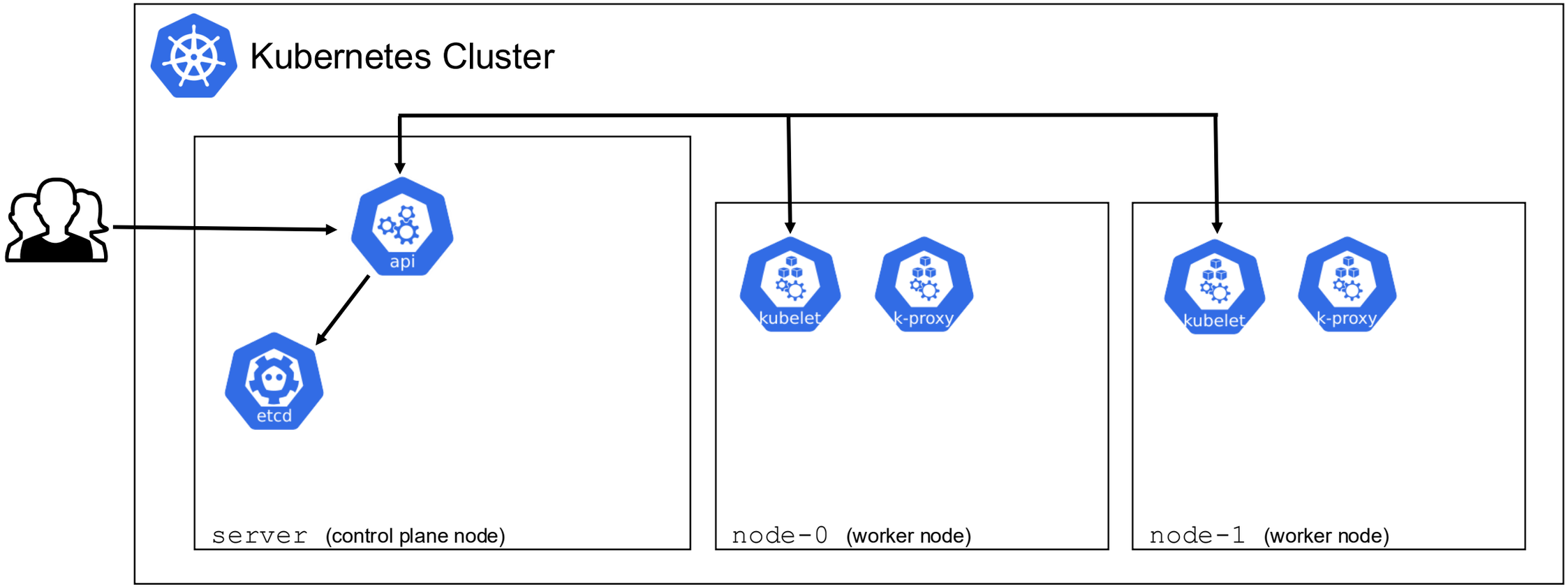
Kubernetes Worker Node

– Kubernetes runs client applications (workloads) by placing containers into Pods on **worker nodes** (virtual or physical machine).

– The **kubelet** is the primary "node agent" that runs on each node. It can register the node with the API server. The kubelet tracks the state of a pod to ensure that all the containers are running using the **Container Runtime Interface (CRI)**. It provides a heartbeat message every few seconds to the control plane.

– The **Kube Proxy (kube-proxy)** routes traffic coming into a node from the services. It forwards requests for work to the correct containers (acts as a L4 load balancer). Often based on *iptables / netfilter*, however it can be replaced using *eBPF* (e.g., Cillium).

Kubernetes Cluster Architecture

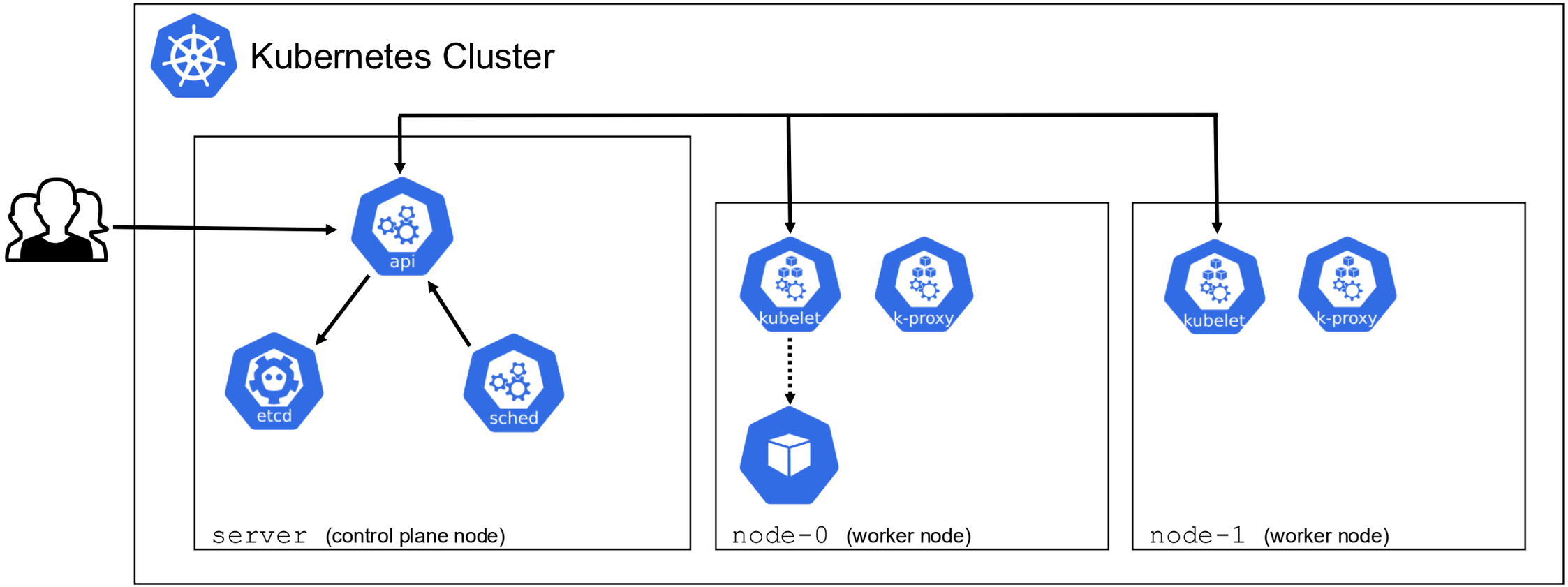


Kubernetes Scheduler

– The **Kubernetes Scheduler (kube-scheduler)** watches for newly created Pods with no assigned node and selects a node for them to run on.

– Factors taken into account for scheduling decisions include:
  - individual and collective resource requirements
  - hardware/software/policy constraints
  - affinity and anti-affinity specifications
  - data locality
  - inter-workload interference

Kubernetes Cluster Architecture



Kubernetes Controllers

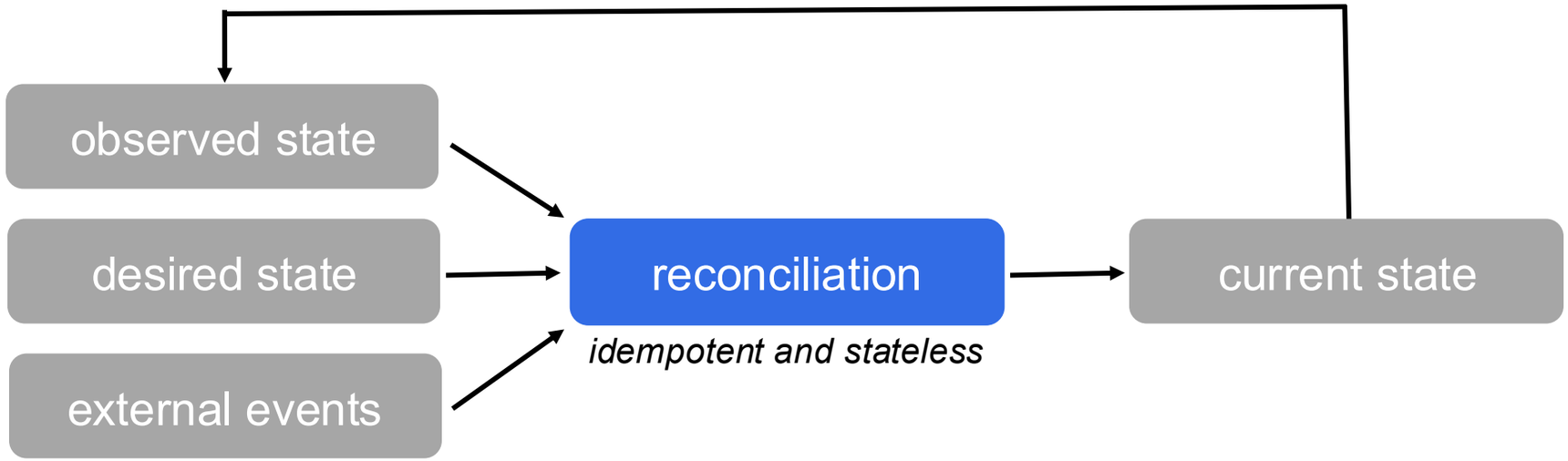
– The **kube-controller-manager** is the control plane component that runs controller processes.

– Logically, each controller is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process.

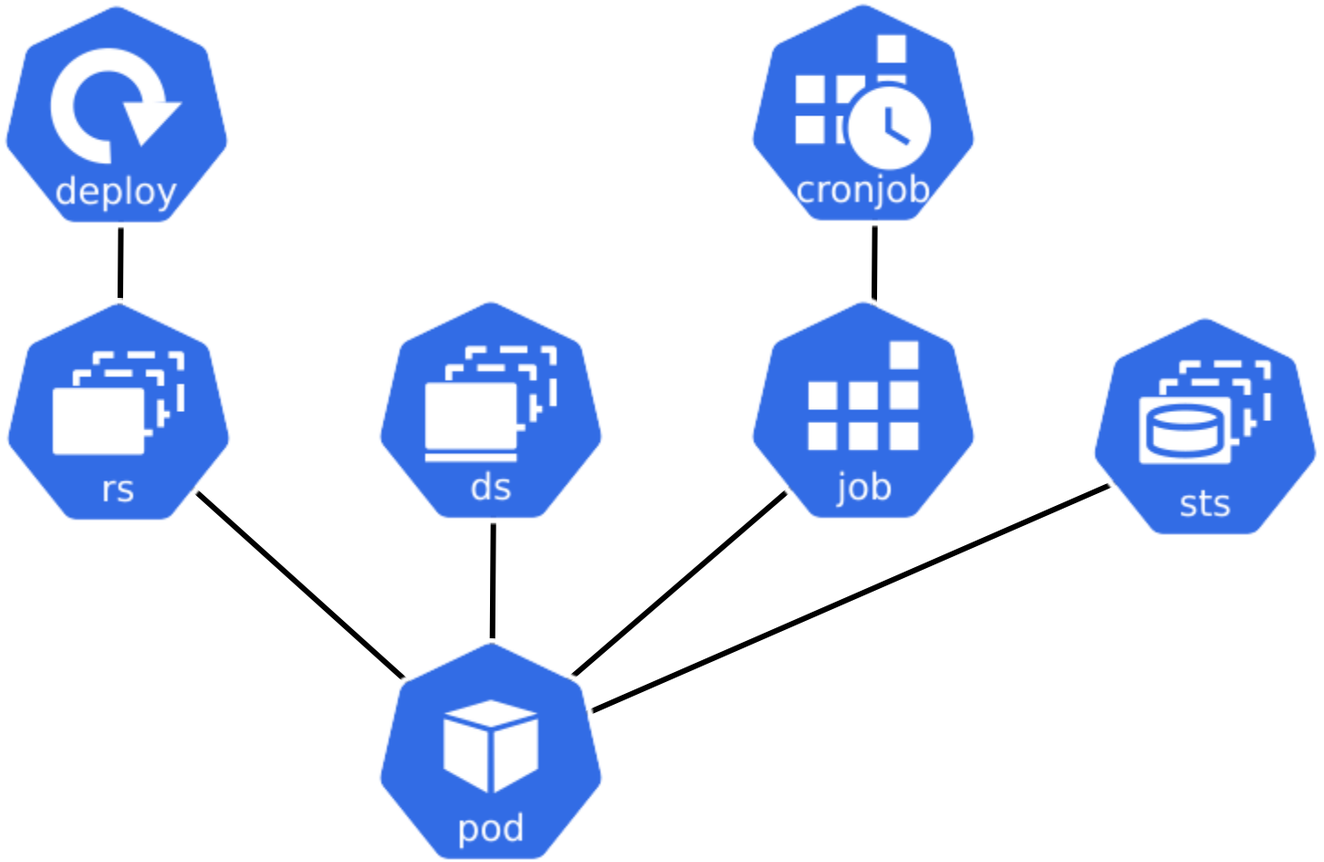
– There are many different types of controllers. Some examples of them are:
  - *Node controller*: Responsible for noticing and responding when nodes go down.
  - *Job controller*: Watches for Job objects that represent one-off tasks, then creates Pods to run those tasks to completion.
  - *EndpointSlice controller*: Populates EndpointSlice objects (to provide a link between Services and Pods).
  - *ServiceAccount controller*: Create default ServiceAccounts for new namespaces.

Kubernetes Controller Pattern

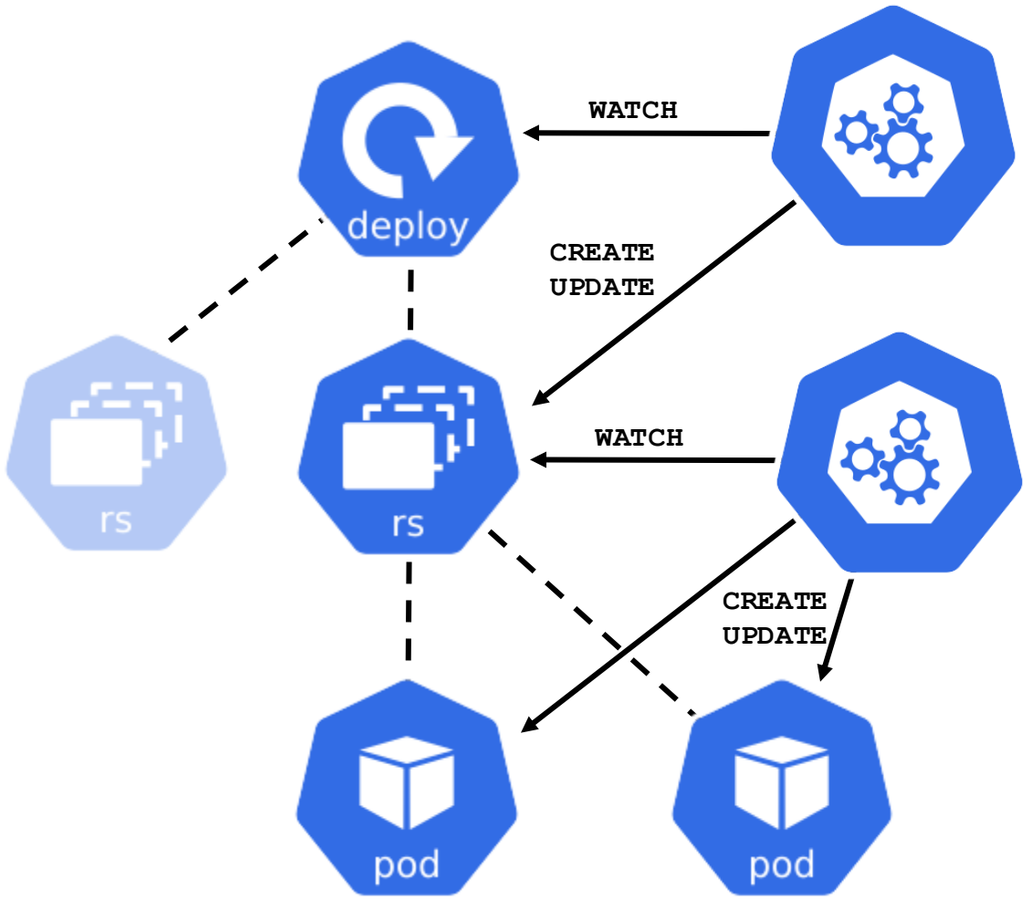
In Kubernetes, controllers are control loops that watch the state of your cluster, then make or request changes where needed. Each controller tries to move the current cluster state closer to the desired state.



Workload Objects



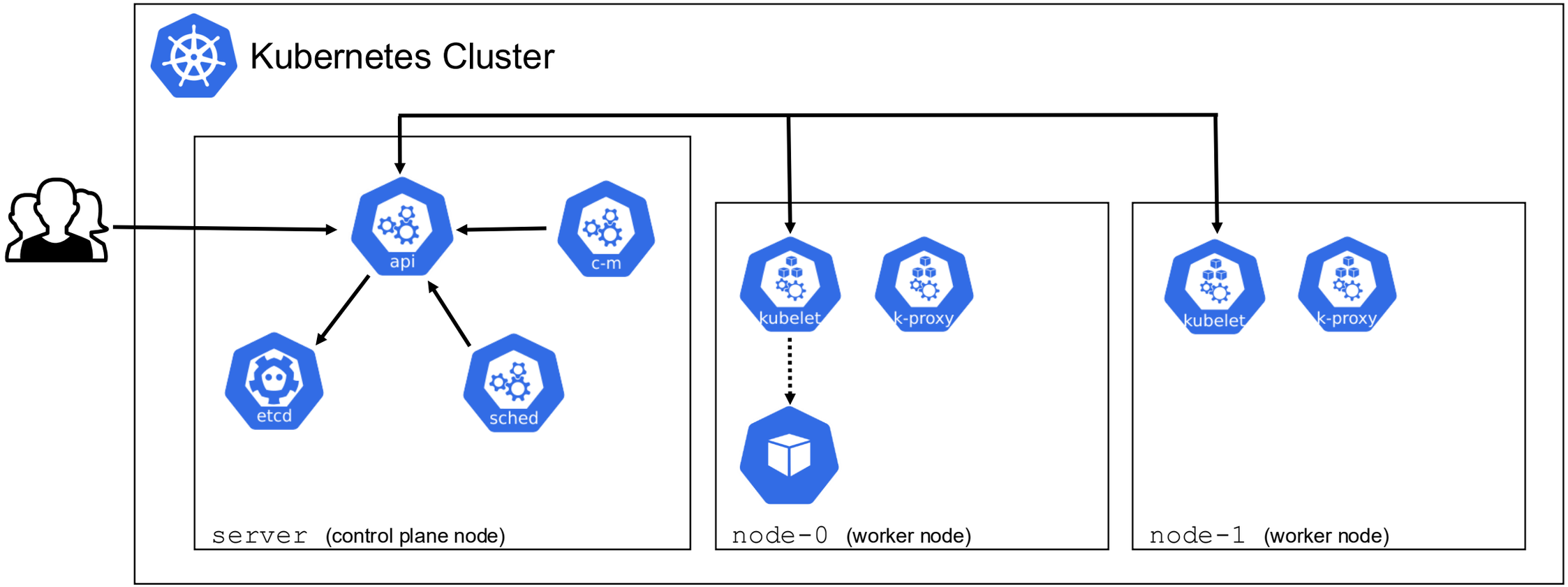
Workload Objects: Deployment

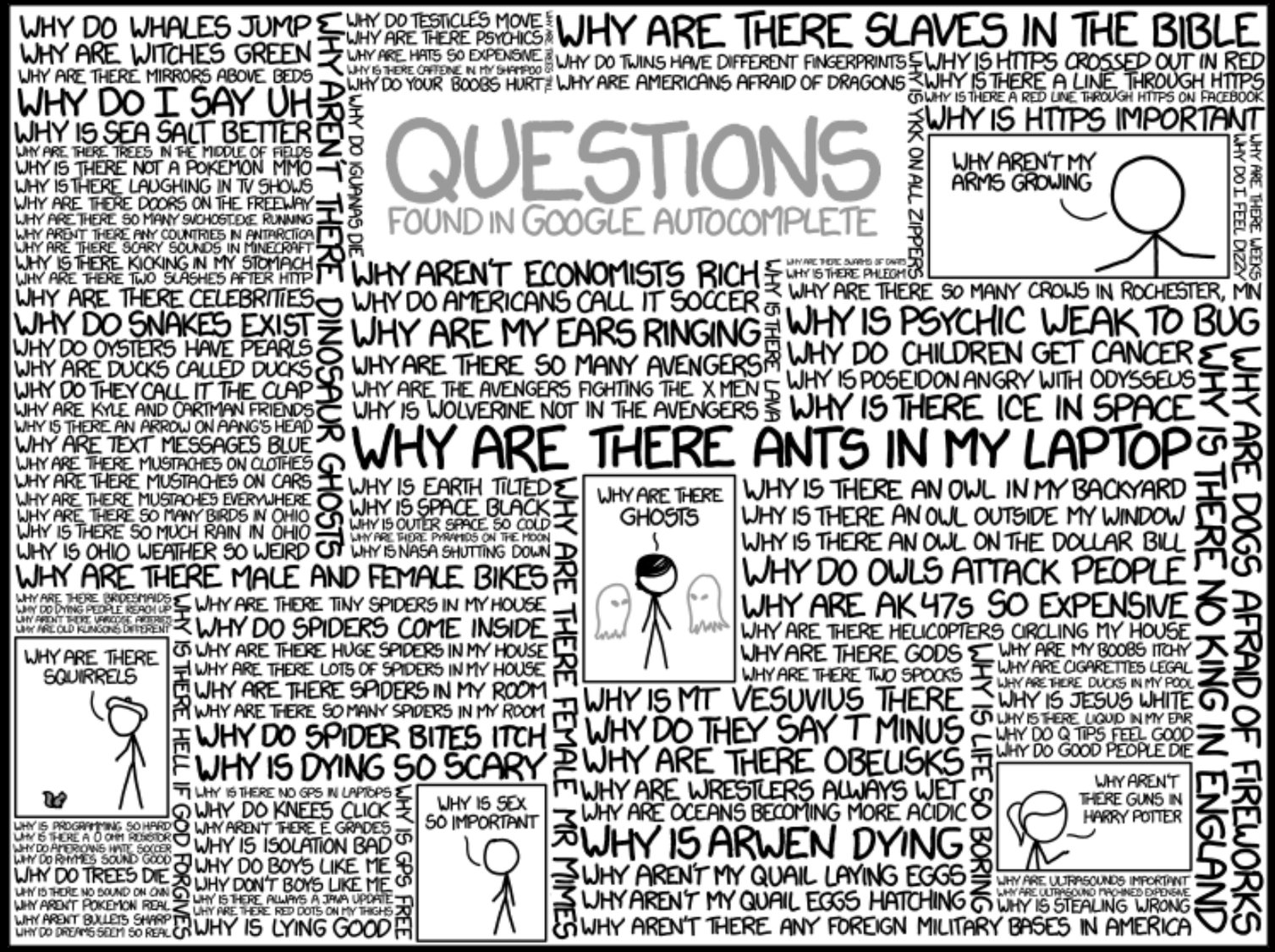


Deployment Controller watches the **Deployment** resource and actual state to the desired state. To achive this, the controller creates, updates or deletes **ReplicaSet** resources.

ReplicaSet Controller maintain a stable set of identical replica Pods running at any given time. In the **ReplicaSet** resource a template of the **Pod** and the desired number of replicas is specified.

Kubernetes Cluster Architecture




QUESTIONS
FOUND IN GOOGLE AUTOCOMPLETE
WHY DO WHALES JUMP
WHY ARE WITCHES GREEN
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SUCHOSTIEXE RUNNING
WHY AREN'T THERE ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY ARE THERE MUSTACHES ON CARS
WHY ARE THERE MUSTACHES EVERYWHERE
WHY ARE THERE SO MANY BIRDS IN OHIO
WHY IS THERE SO MUCH RAIN IN OHIO
WHY IS OHIO WEATHER SO WEIRD
WHY ARE THERE MALE AND FEMALE BIKES
WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
WHY AREN'T THERE VARICOSE AARDVARKS
WHY ARE OLD KLINGONS DIFFERENT
WHY ARE THERE SQUIRRELS
WHY IS PROGRAMMING SO HARD
WHY IS THERE A 0 OHM RESISTOR
WHY DO AMERICANS HATE SOCCER
WHY DO RHYMES SOUND GOOD
WHY DO TREES DIE
WHY IS THERE NO SOUND ON CNN
WHY AREN'T POKEMON REAL
WHY AREN'T BULLETS SHARP
WHY DO DREAMS SEEM SO REAL
WHY DO IGUANAS DIE
WHY DO TESTICLES MOVE
WHY ARE THERE PSYCHICS
WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY DO YOUR BOOBS HURT
WHY ARE THERE SLAVES IN THE BIBLE
WHY DO TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE AMERICANS AFRAID OF DRAGONS
WHY IS HTTPS CROSSED OUT IN RED
WHY IS THERE A LINE THROUGH HTTPS
WHY IS THERE A RED LINE THROUGH HTTPS ON FACEBOOK
WHY IS YKK ON ALL ZIPPERS
WHY IS HTTPS IMPORTANT
WHY AREN'T MY ARMS GROWING
WHY ARE THERE WEEKS
WHY DO I FEEL DIZZY
WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY ARE THERE SO MANY AVENGERS
WHY ARE THE AVENGERS FIGHTING THE X MEN
WHY IS WOLVERINE NOT IN THE AVENGERS
WHY ARE THERE SWARMS OF DAVES
WHY IS THERE PHLEGM
WHY IS THERE LAVA
WHY ARE THERE SO MANY CROWS IN ROCHESTER, MN
WHY IS PSYCHIC WEAK TO BUG
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE
WHY ARE THERE ANTS IN MY LAPTOP
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY ARE THERE GHOSTS
WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE AK 47s SO EXPENSIVE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOOKS
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPIDER BITES ITCH
WHY IS DYING SO SCARY
WHY IS THERE NO GPS IN LAPTOPS
WHY DO KNEES CLICK
WHY AREN'T THERE E GRADES
WHY IS ISOLATION BAD
WHY DO BOYS LIKE ME
WHY DON'T BOYS LIKE ME
WHY IS THERE ALWAYS A JAVA UPDATE
WHY ARE THERE RED DOTS ON MY THIGHS
WHY IS LYING GOOD
WHY IS SEX SO IMPORTANT
WHY IS GPS FREE
WHY IS MT VESUVIUS THERE
WHY DO THEY SAY T MINUS
WHY ARE THERE OBELISKS
WHY ARE WRESTLERS ALWAYS WET
WHY ARE OCEANS BECOMING MORE ACIDIC
WHY IS ARWEN DYING
WHY AREN'T MY QUAIL LAYING EGGS
WHY AREN'T MY QUAIL EGGS HATCHING
WHY AREN'T THERE ANY FOREIGN MILITARY BASES IN AMERICA
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO GOOD PEOPLE DIE
WHY AREN'T THERE GUNS IN HARRY POTTER
WHY ARE ULTRASOUNDS IMPORTANT
WHY ARE ULTRASOUND MACHINES EXPENSIVE
WHY IS STEALING WRONG
WHY IS LIFE SO BORING
WHY ARE DOGS AFRAID OF FIREWORKS
WHY IS THERE NO KING IN ENGLAND
WHY ARE THERE FEMALE MR MIMES
WHY IS THERE HELL IF GOD FORGIVES

Summarizing Questions

- What are components of the control plane?
- What are components of the worker node?
- How is independence between the cloud infrastructure and the running kubernetes cluster achieved?
- Why is backupping the etcd important for the cluster?
- What is the difference between a container and a pod?
- What are use cases docker swam can not fullfil but kubernetes can?
- Describe in your own words, what the purpose of the kube scheduler is?